

Lab 16: Deploy CockroachDB on Kubernetes with `cockroach-operator` and `kind` (90 min)

Learning Objectives

By the end of this lab you will be able to:

- Stand up a multi-node Kubernetes cluster locally with `kind` and deploy CockroachDB via the operator
- Read the `CrdbCluster` CRD and change resources, replica count, and version through it
- Perform a rolling version upgrade and a scale-out with zero downtime
- Configure pod anti-affinity so replicas never share a failure domain
- Kill a pod and a node and watch Raft and the StatefulSet recover
- Compare the operator, the Helm chart, and manual StatefulSets — and pick one with reasons

Prerequisites

- Docker with at least **8 GB** of memory free (a full run was measured at 7.6 GB)
- `kind` and `kubectl` — the only two tools this course installs locally, because `kind` drives the Docker daemon itself and cannot run in a container
- **Docker Desktop** (or Docker Engine) running — there is no `cockroach` binary to install

Free the memory first. `kind` runs four Kubernetes nodes and then three CockroachDB pods inside them. On a 12 GB machine that does not fit alongside the lab cluster, so shut it down before you start:

```
scripts/crdb down
```

Skip this and the symptom is not a clear error — pods sit in `Pending` or get OOM-killed mid-rollout, which reads like an operator bug.

```
kind version && kubectl version --client && docker info | grep -i "total memory"
```

Setup — a Multi-Node kind Cluster

```
mkdir -p /tmp/lab16 && cd /tmp/lab16

cat > kind-config.yaml <<'YML'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
    labels: {topology.kubernetes.io/zone: zone-a}
  - role: worker
    labels: {topology.kubernetes.io/zone: zone-b}
  - role: worker
    labels: {topology.kubernetes.io/zone: zone-c}
YML

kind create cluster --name lab16 --config kind-config.yaml
kubectl get nodes --show-labels | cut -c1-120
```

Three workers with distinct zone labels — that's what makes the anti-affinity in Part D real rather than decorative.

Tasks

Part A: Install the Operator (15 min)

1. CRDs and operator:

```
kubectl apply -f https://raw.githubusercontent.com/cockroachdb/cockroach-operator/master/
install/crds.yaml
kubectl apply -f https://raw.githubusercontent.com/cockroachdb/cockroach-operator/master/
install/operator.yaml
```

2. **Wait for it — twice.** The Pod being ready is not the same as the operator's admission webhook being reachable, and applying a `CrdbCluster` in that gap fails with `failed calling webhook "mcrdbcluster.kb.io" ... connection refused`:

```
kubectl -n cockroach-operator-system rollout status deploy/cockroach-operator-manager --
timeout=180s
kubectl -n cockroach-operator-system get pods
```

```
# The webhook Service must have at least one ready endpoint before you apply anything.
kubectl -n cockroach-operator-system get endpoints cockroach-operator-webhook-service -w
```

This is a general Kubernetes lesson, not an operator quirk: **any** operator with a mutating or validating webhook rejects resources until its Service has endpoints. If your CI applies a custom resource immediately after installing an operator, it will flake.

3. Look at what the CRD gives you:

```
kubectl get crd crdbclusters.crdb.cockroachlabs.com -o yaml | head -40
kubectl explain crdbcluster.spec | head -40
```

Field	Controls
<code>spec.nodes</code>	Cluster size — change it to scale
<code>spec.cockroachDBVersion</code>	Version — change it to upgrade
<code>spec.resources</code>	CPU/memory requests and limits per pod
<code>spec.dataStore.pvc</code>	Storage class and size per node
<code>spec.tlsEnabled</code>	Operator-managed certs via cert-manager or self-signed
<code>spec.affinity / topologySpreadConstraints</code>	Failure-domain placement
<code>spec.additionalArgs</code>	Extra <code>cockroach start</code> flags (e.g. <code>--locality</code>)

Part B: Deploy the Cluster (15 min)

1. The manifest — `/tmp/lab16/crdb.yaml` :

```

apiVersion: crdb.cockroachlabs.com/v1alpha1
kind: CrdbCluster
metadata:
  name: crdb
  namespace: default
spec:
  datastore:
    pvc:
      spec:
        accessModes: [ReadWriteOnce]
        resources:
          requests:
            storage: 2Gi
          volumeMode: Filesystem
  resources:
    requests: {cpu: "500m", memory: 1Gi}
    limits: {cpu: "1", memory: 2Gi}
  tlsEnabled: true
  cockroachDBVersion: v23.2.5
  nodes: 3
  additionalLabels:
    app: crdb
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
      labelSelector:
        matchLabels:
          app.kubernetes.io/instance: crdb

```

Memory limits matter more here than anywhere else. CockroachDB sizes its cache from what it believes the machine has. Under a cgroup limit, always set `--cache` and `--max-sql-memory` as *percentages* (the operator does this) and leave headroom — a pod OOM-killed mid-Raft-append is a much worse outcome than a slightly smaller cache.

2. Apply and watch it come up:

```

kubectl apply -f /tmp/lab16/crdb.yaml
kubectl get pods -w          # Ctrl+C once all 3 are Running

```

3. Initialize and connect:

```

kubectl apply -f https://raw.githubusercontent.com/cockroachdb/cockroach-operator/master/
examples/client-secure-operator.yaml
kubectl exec -it cockroachdb-client-secure -- ./cockroach sql \
  --certs-dir=/cockroach/cockroach-certs --host=crdb-public

```

```

SELECT node_id, address, is_live FROM crdb_internal.gossip_nodes;
SELECT count(*) FROM crdb_internal.ranges_no_leases;

```

4. Map Kubernetes objects to CockroachDB concepts:

```
kubectl get statefulset,svc,pvc,pdb
```

Kubernetes object	Role
StatefulSet <code>crdb</code>	Stable pod identities (<code>crdb-0</code> , <code>crdb-1</code> , ...) — a node's identity must survive restarts
Headless service <code>crdb</code>	Peer discovery for <code>--join</code>
Service <code>crdb-public</code>	Client entry point
PVC per pod	The store. Deleting it deletes that node's data
PodDisruptionBudget	Stops <code>kubectl drain</code> from taking quorum with it

5. Open the DB Console:

```
kubectl port-forward svc/crdb-public 8080:8080
```

Then <https://localhost:8080> (self-signed cert — accept the warning).

Part C: Rolling Upgrade and Scale-Out (20 min)

1. Generate load so you can prove "zero downtime" instead of asserting it:

```
kubectl exec -it cockroachdb-client-secure -- ./cockroach workload init kv \
  --drop 'postgres://root@crdb-public:26257/defaultdb?sslmode=verify-full&sslrootcert=/
cockroach/cockroach-certs/ca.crt&sslcert=/cockroach/cockroach-certs/
client.root.crt&sslkey=/cockroach/cockroach-certs/client.root.key'

kubectl exec -it cockroachdb-client-secure -- ./cockroach workload run kv \
  --duration=10m --concurrency=8 --tolerate-errors \
  'postgres://root@crdb-public:26257/defaultdb?sslmode=verify-full&sslrootcert=/
cockroach/cockroach-certs/ca.crt&sslcert=/cockroach/cockroach-certs/
client.root.crt&sslkey=/cockroach/cockroach-certs/client.root.key' &
```

2. Scale out — one field:

```
kubectl patch crdbcluster crdb --type=merge -p '{"spec":{"nodes":5}}'
kubectl get pods -w
```

3. Watch the data rebalance onto the new nodes:

```
SELECT node_id, count(*) AS replicas
FROM crdb_internal.ranges_no_leases, unnest(replicas) AS node_id
GROUP BY node_id ORDER BY node_id;
```

Run it a few times over a couple of minutes. The allocator moves replicas onto the new nodes without anyone telling it to.

4. Rolling version upgrade:

```
kubectl patch crdbcluster crdb --type=merge -p '{"spec":{"cockroachDBVersion":"v23.2.6"}}'
kubectl rollout status statefulset/crdb --timeout=600s
```

Watch the workload output during the roll. Errors should be zero (or retried, with `--tolerate-errors`), because at any moment only one node is down and quorum holds.

5. **Finalize the upgrade** — a major-version upgrade stays reversible until you finalize:

```
SHOW CLUSTER SETTING version;
-- After validating, for a major upgrade:
-- SET CLUSTER SETTING version = crdb_internal.node_executable_version();
```

6. **Record the drill:**

Operation	Duration	Client errors	Notes
Scale 3 → 5			
Rebalance to steady state			
Rolling upgrade			

Part D: Failure Domains and Recovery (20 min)

1. **Confirm placement honours the zone spread:**

```
kubectl get pods -o custom-columns=POD:.metadata.name,NODE:.spec.nodeName --sort-by=.spec.nodeName
kubectl get nodes -L topology.kubernetes.io/zone
```

2. **Tell CockroachDB about the topology**, not just Kubernetes. Without `--locality`, the allocator does not know two pods share a zone, and may place all three replicas of a range in one failure domain:

```
kubectl patch crdbcluster crdb --type=merge -p \
'{"spec":{"additionalArgs":["--locality=zone=$(POD_ZONE)"]}}'
```

In a real deployment the zone comes from the downward API or an init container reading the node label. This is the single most commonly skipped step in Kubernetes deployments, and it silently defeats the whole point of running three zones.

3. **Kill a pod:**

```
kubectl delete pod crdb-2
kubectl get pods -w
```

The StatefulSet recreates it with the *same identity* and the *same PVC*, so it rejoins with its data instead of re-replicating from scratch.

4. **Kill a whole node:**

```
docker stop lab16-worker2
kubectl get nodes
```

```
SELECT node_id, is_live FROM crdb_internal.gossip_nodes;
SELECT count(*) FROM crdb_internal.ranges_no_leases; -- still answers: quorum held
```

Watch under-replicated ranges in the DB Console. After `server.time_until_store_dead` (default 5 min) the cluster starts re-replicating.

5. Bring it back:

```
docker start lab16-worker2
kubectl get nodes -w
```

6. Decommission properly — never just delete a StatefulSet pod and its PVC:

```
kubectl exec -it cockroachdb-client-secure -- ./cockroach node status \
  --certs-dir=/cockroach/cockroach-certs --host=crdb-public

# Correct order: decommission (moves replicas off), THEN scale down
kubectl exec -it cockroachdb-client-secure -- ./cockroach node decommission 5 \
  --certs-dir=/cockroach/cockroach-certs --host=crdb-public --wait=all

kubectl patch crdbcluster crdb --type=merge -p '{"spec":{"nodes":4}}'
```

The failure mode to avoid: scaling down first, then decommissioning. Removing the pod takes its replicas offline instantly; the cluster then has to re-replicate under duress. Decommission moves replicas off *while the node is still serving*, which is why it is slower and correct.

Part E: Operator vs Helm vs Manual (10 min)

Approach	Best for	Handles for you	You still own
cockroach-operator	Most production K8s deployments	Certs, rolling upgrades, scaling, decommission ordering	Locality flags, storage class, monitoring
Helm chart	Existing Helm/GitOps pipelines	Templating, values-driven config	Upgrade ordering, cert lifecycle, decommission
Manual StatefulSet	Unusual requirements, air-gapped, full control	Nothing	Everything — including the ordering mistakes above

1. Look at the Helm alternative:

```
# helm in a container – nothing to install. The two mounts matter: without
# them `repo add` writes into a container that is deleted a moment later, and
# `show values` then reports no such repository.
mkdir -p ~/.config/helm ~/.cache/helm
alias helm='docker run --rm -v "$HOME/.config/helm:/root/.config/helm" -v "$HOME/.cache/helm:/root/.cache/helm" alpine/helm'
helm repo add cockroachdb https://charts.cockroachdb.com/
helm show values cockroachdb/cockroachdb | head -60
```

2. The multi-region shape (for discussion — it needs three real clusters):

- One `CrdCluster` per region, each with region-specific `--locality`
- Cross-cluster networking so pods can gossip (service mesh, VPC peering, or LB per region)
- `ALTER DATABASE ... SET PRIMARY REGION / ADD REGION` from Lab 7
- Survival goal set to `REGION` — which needs ≥ 3 regions and ≥ 5 replicas

3. Production checklist for a K8s deployment:

☐

`--locality` reflects real failure domains (zone, region)

☐

`topologySpreadConstraints` or anti-affinity prevents co-location

☐

`PodDisruptionBudget` protects quorum from voluntary disruptions

☐

Storage class is SSD-backed with `volumeBindingMode: WaitForFirstConsumer`

☐

Resource requests == limits for CPU-sensitive workloads (guaranteed QoS)

☐

Prometheus `ServiceMonitor` scraping `/_status/vars` (Lab 9)

☐

Scheduled backups to object storage, and a completed restore drill (Lab 11)

☐

`cert-manager` or the operator managing cert rotation (Lab 12)

☐

A documented decommission-then-scale-down runbook

Cleanup

```
kind delete cluster --name lab16
```

Lab 16 Deliverables

□ **4-node kind cluster** with zone labels, operator installed □ **3-node CockroachDB** deployed via the CRD with TLS and topology spread □ **Scale-out** to 5 nodes with rebalancing observed □ **Rolling upgrade** completed under live load with client errors recorded □ **Pod and node failures** survived; identity and PVC reattachment understood □ **Correct decommission ordering** performed □ **Deployment comparison** with a defended choice, plus a production checklist

Challenge Exercises

1. **cert-manager integration.** Replace the operator's self-signed certs with cert-manager issued certs and rotate them without downtime (Lab 12's SIGHUP applies here too).
2. **Prove the anti-affinity.** Remove `topologySpreadConstraints`, redeploy, and check whether replicas of the same range land in one zone. How would you detect this in production before an outage does it for you?
3. **Break decommissioning on purpose.** Scale from 5 → 3 by patching `nodes` *without* decommissioning first. What does `SHOW RANGES` report? How do you recover?
4. **Backups from inside K8s.** Add a `CronJob` that runs `BACKUP INTO 's3://...'` and alerts on failure. Where do the credentials live, and why not in the manifest?

Reference

Command	Purpose
<code>kind create cluster --config</code>	Multi-node local Kubernetes
<code>kubectl apply -f ../operator.yaml</code>	Install the CockroachDB operator
<code>kubectl explain crdbcluster.spec</code>	Discover CRD fields
<code>kubectl patch crdbcluster crdb --type=merge -p '{"spec":{"nodes":N}}'</code>	Scale
<code>kubectl patch ... '{"spec":{"cockroachDBVersion":"vX.Y.Z"}}'</code>	Rolling upgrade
<code>kubectl rollout status statefulset/crdb</code>	Watch the roll
<code>cockroach node decommission N --wait=all</code>	Move replicas off before removing a node
<code>topologySpreadConstraints</code>	Keep replicas out of the same failure domain
<code>--locality=zone=...</code>	Tell CockroachDB about the topology